



Welcome to this course on creating a walking simulator in the Godot engine. My name is Daniel Buckley, and I'll be your instructor for this course. We will be creating essential systems that you need to develop a walking simulator, mainly a player controller and an interaction controller. These systems are designed to be modular, which means you can easily integrate them into your game projects.

Player Controller

The player controller will handle character movement using the WASD keys, running, jumping, and camera look around. We will also implement a movement smoothing system to make the movement feel more natural. This controller is versatile and can be used in almost any 3D game that you develop.

- Movement with WASD keys
- Ability to run and jump
- Look around with the camera
- Movement smoothing

Interaction Controller

The interaction controller works by shooting out a raycast from the center of your camera to whatever object you're looking at. If the raycast hits an object, you can press 'E' to interact with it. A text box will appear on the screen to provide feedback about what you're interacting with.

- Shoots a raycast from the center of the camera
- Interact with objects by pressing 'E'
- Displays a text box for interaction feedback

To make an object interactable, you only need to extend from a specific class and create a single function. This makes the interaction system easy to implement in your game.

Prerequisites

While we will be covering topics such as vectors and dot products, a basic understanding of Godot and GDScript is all that's required for this course. You should be familiar with creating nodes, variables, and functions. More complex topics like vector math and class inheritance will be explained during the course.

About Zenva

Zenva is an online learning platform with over 1 million students. We offer a wide range of courses for beginners and those looking to learn something new. Our courses are versatile, allowing you to learn in the way that suits you best. You can watch online tutorials, read lesson summaries, or follow along with the course files.

Let's get started with the first lesson.

Course Updated to Godot 4.5

We've updated the project files to Godot version 4.5 for this course – the latest stable release.

Please make sure NOT to use newer versions of the software than what we recommend, as the course material provided might not work as expected.

How to Install Version 4.5

You can download the most recent version of Godot by heading to the Godot website (<https://godotengine.org/>) and clicking the “Download Latest” button on the front page. This will automatically take you to the download page for your operating system.



Once on the download page, just click the option for **Godot 4.5**. This will download the Godot engine to your local computer. From there, unzip the file and simply click on the application launcher – no further installation steps are required!

Download Godot 4 for Windows

**Godot Engine****4.5**

x86_64 - 15 September 2025

**Godot Engine – .NET****4.5**

x86_64 · C# support - 15 September 2025

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?

Download Godot 4 for macOS

 **Godot Engine** **4.5**

arm64 (Apple Silicon) · x86_64 (Intel) · 15 September 2025

 **Godot Engine – .NET** **4.5**

arm64 (Apple Silicon) · x86_64 (Intel) · C# support · 15 September 2025

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?

If Godot fails to automatically select the correct operating system for you, you can scroll down to the “Supported platforms” section on the download page to manually select the download version you would like to install:

Supported platforms

 **Android**  **Linux**  **macOS**  **Windows**  **Web Editor**



The instructions in the video for this particular lesson have been updated, please see the lesson notes below for the corrected version.

In this tutorial, we will be setting up a walking simulator project using the Godot editor. This involves creating a new project, setting up our environment, and importing assets for our scene. We will also be creating our player controller and interaction controller in the following lessons.

Creating a New Project

The first step in creating our walking simulator is to set up a new project in the Godot project manager. We will name this project "Walking Simulator". Once the project is created, we can begin to set up our environment within the Godot editor.

Setting Up the Environment

We start by creating a root node for our scene, selecting 3D scene and renaming it to "main". This node is then saved to our file system.

Importing Assets

Next, we import assets for our scene. These assets can be found in a zip file in the course files tab. Once the zip file is downloaded and extracted, it will contain a models folder with various 3D models and textures. We simply drag this models folder into our file system in Godot, and the editor will automatically import all the assets.

Setting Up the Terrain

With our assets imported, we can begin setting up our terrain. We create a static body 3D node for our base terrain and position it at the origin (0,0,0). We also apply a terrain material to our terrain model. To prevent our player from falling through the ground, we need to create a collider for our terrain. This is done by selecting the terrain mesh and creating a "Trimesh" collision sibling.

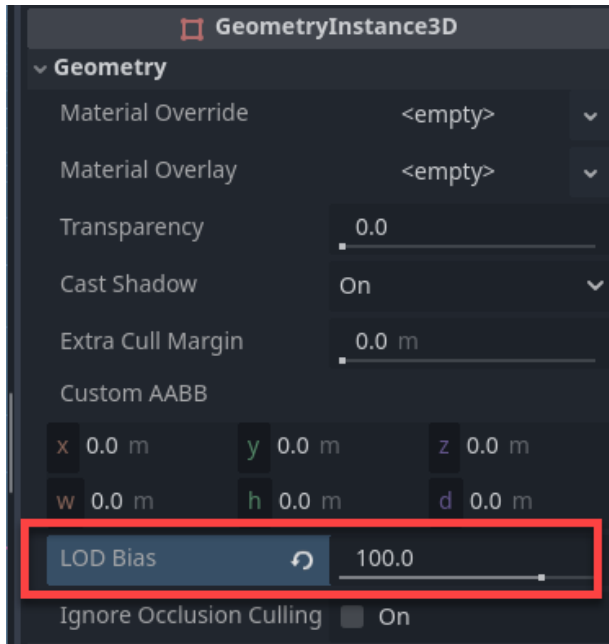
Decorating the Scene

Now that we have our basic terrain, we can begin to decorate our scene. We can add trees and rocks by dragging in the corresponding .tcn files into our scene. These models have already been set up with static meshes and collisions. We can move and duplicate these models around our scene to create a visually appealing environment.

The following instructions have been updated, and differ from the video:

If you're having issues with the tree models flickering when zooming in and out in the scene, apply the following fix:

Go into the Tree scene and select the 'TreeObj' node. In the inspector, set the **LOD Bias** property to a high value, say 100 for example.



That should fix any issues you have with flickering trees!

Adding Lighting and Fog

To make our environment more realistic, we can add a directional light and enable shadows. We can also add fog by creating a world environment node and enabling the fog property. The color of the fog can be adjusted to suit the mood of our scene.

Next Steps

In the next lessons, we will be creating the player controller and interaction controller for our walking simulator. These controllers will allow our player to move around the scene and interact with objects in the world. Stay tuned for more!



In this tutorial, we will be setting up our player controller in the Godot game engine. This involves creating inputs that control actions in the game, such as moving forward, moving back, jumping, and sprinting. We will also set up the player node and add a camera for the player's point of view.

Creating Inputs

The first step is to create our inputs, which will determine which buttons perform which actions. To do this:

1. Go to Project settings.
2. Go to the Input Map tab.
3. Create actions and assign inputs to those actions by going to the "Add new action" text box.
4. Assign a key or an input to the action using the plus icon on the right-hand side.

Here are the actions we will be creating and the keys we will assign to them:

- Move forward – W
- Move back – S
- Move left – A
- Jump – Spacebar
- Sprint – Left Shift
- Interact – E

Note that you can assign multiple inputs to each action, and you can also use mouse buttons and joystick buttons.

Creating the Player Node

Next, we will create our player node. This will be a `CharacterBody3D` node, which conveniently handles movement, collision, and velocity calculations for us. We'll name this node "player".

We also need to add child nodes to our player. These include a `CollisionShape3D` node, which acts as the player's collider, and a `Camera3D` node, which provides the player's point of view. The Collider should be a capsule shape, as this best fits a humanoid character and allows for smooth movement over surfaces. Adjust the radius and height of the collider to fit your character, and ensure the bottom of the collider is touching the ground.

Adding a Camera

The camera should be placed at eye level for the player. Once you've positioned the camera, enable "current" on it so that it renders the player's view. Save the scene, and you should now be able to see from the player's perspective. However, at this stage, you won't be able to move or look around yet.

In the next tutorial, we will continue our player controller setup by enabling movement, jumping, sprinting, and looking around. With these features, you will have a versatile player controller that you can use in any project.



In this lesson, we will start scripting our player in Godot. We'll be creating a new script and setting up the necessary variables and functions for our player character. We will also organize our variables in the inspector using export groups. Let's get started.

Creating the Player Script

First, select the player node, in this case, the character Body 3D. Go down to the script section and create a new script called 'Player'. The newly created script may come with a bunch of default code. Delete everything to start with a blank script.

At the beginning of the script, specify that it extends from 'Character Body 3D'. This means that we'll have access to a lot of our character body functionality, such as setting the velocity and checking if we are standing on the ground or not.

Next, create a class name for this script. This will be handy because you might want to reference it directly in another script later on. We'll call it 'Player Controller'.

```
class_name PlayerController
extends CharacterBody3D
```

Creating Variables

Now, let's create some variables for our player. The first variable will be our max speed. This is going to be of type float and set to 4 by default. This variable will be accessible and modifiable inside the inspector. To do this, add an export tag at the start.

```
@export
var max_speed : float = 4.0
```

Repeat this process to create several other variables. These include:

- Acceleration: The speed at which the player moves towards the max speed.
- Braking: The speed at which the player slows down when the keyboard is released.
- Air Acceleration: The speed at which the player moves when in the air.
- Jump Force: The amount of force applied upwards when the player jumps.
- Gravity Modifier: Modifies the project's default gravity settings.
- Max Run Speed: The speed at which the player moves when running.
- Is Running: A boolean variable that checks if the player is running or not.

```
@export
var acceleration : float = 20.0
@export
var braking : float = 20.0
@export
var air_acceleration : float = 4.0
@export
var jump_force : float = 5.0
@export
var gravity_modifier : float = 1.5
```



```
@export
var max_run_speed : float = 6.0
var is_running : bool = false
```

We also need to create some variables for our camera, including:

- Look Sensitivity: The sensitivity of the camera movement.
- Camera Look Input: Keeps track of the speed and direction that the mouse is moving.

```
@export
var look_sensitivity : float = 0.005
var camera_look_input : Vector2
```

Finally, we create two 'on ready' variables. These variables are assigned when the node is initialized. These include:

- Camera: Gets the Camera3D node at the start of the game.
- Gravity: Gets the default gravity and multiplies it by our gravity modifier.

```
@onready var camera : Camera3D = get_node("Camera3D")
@onready var gravity : float = ProjectSettings.get_setting("physics/3d/default_gravity") * gravity_modifier
```

Creating Functions

Now, let's create the functions for our script. These include:

- `_ready`: This function is called when the node is initialized.
- `_physics_process`: This function is called every physics frame, and is where we'll be modifying and adjusting everything related to the physics engine.
- `_unhandled_input`: This function is called whenever we create an input or move our mouse.

```
func _ready():
    pass
```

```
func _physics_process(delta):
    pass
```

```
func _unhandled_input(event):
    pass
```

Organizing Variables Using Export Groups

To make our script more organized, we can put our variables in export groups. This will categorize



them into dropdowns in the inspector, making them easier to manage. For example, we can create an export group for our movement variables:

```
@export_group("Movement")
```

And another one for our camera variables:

```
@export_group("Camera")
```

In the inspector, you will now see two dropdowns: one for all of our movement variables and one for all of our camera-related variables.

That's it for this lesson! We've set up our player controller script with variables and functions. In the next lesson, we'll start filling in these functions to add functionality to our player.



In this lesson, we will be building upon our script to control player movement and camera rotation. We'll start by working with the unhandled input function, which gets called whenever an input is detected, such as keyboard or mouse movement. We will then move on to the physics process function to handle movement and camera rotation. Finally, we will be adding a feature to lock the mouse to the screen, which is a common feature in first-person games.

Unhandled Input Function

The unhandled input function is used to handle inputs that the system does not automatically process. In our case, we are interested in mouse motion events. We will check if the event parameter is a mouse motion event. If it is, we will set our camera look input variable to the relative movement of the mouse.

```
if event is InputEventMouseMotion:  
    camera_look_input = event.relative
```

This will set our mouse movement to be the camera look input variable.

Physics Process Function

The physics process function is where we will handle the majority of our script. We will start by accessing the velocity variable of our character body 3D node. The velocity is a vector three that we can set to determine the direction and speed of movement. For instance, setting the velocity to vector three forward will move us in the forward direction.

```
velocity = Vector3.FORWARD
```

However, setting the velocity alone will not move our character body. To actually move our character body, we need to call the `move_and_slide` function, which applies the set velocity to our player.

```
move_and_slide(velocity)
```

With this, we should be able to move our character forward. But we want to move based on our keyboard inputs, not just a static vector. To achieve this, we will use the `get_vector` function, which takes in four input actions and returns a vector for us to use.

```
var move_input = Input.get_vector("move_left", "move_right", "move_forward", "move_back")
```

After getting the move input, we can set our velocity to this input. However, since velocity is a `Vector3` and `move_input` is a `Vector2`, we need to convert `move_input` to a `Vector3`. We can do this by setting the x and z values of our velocity to the x and y values of our `move_input`, respectively. We will also multiply the velocity by our max speed to control the speed of our character.



```
velocity = Vector3(move_input.x, 0, move_input.y) * max_speed
```

Camera Rotation

When dealing with camera rotation, we will only be moving our camera up and down when we move our mouse. To look left and right, we will be rotating our entire player body instead of the camera. We can achieve this by using the `rotate_y` function for horizontal rotation and the `rotate_x` function for vertical rotation.

```
rotate_y(-camera_look_input.x * look_sensitivity)  
camera.rotate_x(-camera_look_input.y * look_sensitivity)
```

We also need to clamp the camera's x rotation to prevent our player from flipping upside down when looking too far up or down. We can use the `clamp` function to restrict the rotation between -1.5 and 1.5, which is equivalent to -90 degrees and 90 degrees.

```
camera.rotation.x = clamp(camera.rotation.x, -1.5, 1.5)
```

Locking the Mouse to the Screen

Finally, we will lock the mouse to the screen by setting the mouse mode to `MOUSE_MODE_CAPTURED` in the `ready` function. This will make our mouse invisible and allow us to look around freely.

```
Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
```

In the next lesson, we will be improving our player movement by making it relative to the camera direction. This will allow our player to move in the direction they are looking at, providing a more immersive gaming experience.



In this lesson, we will continue working on our player controller in Godot Engine. We will set up the player character to move relative to the camera's forward direction. After that, we will add jumping functionality to the player character and apply gravity. Let's get started.

Setting Up Player Movement Relative to Camera's Forward Direction

First, we need to modify our physics process function, where our movement code is running. We will calculate our move direction based on our transform basis – our orientation in the world. This takes into account our rotation and which direction is forward, backward, left, and right. Then we will normalize this vector to have a magnitude of one.

After calculating the move direction, we can set our velocity to be equal to our move direction multiplied by our max speed.

Here is how the code looks:

```
# Calculate move direction
var move_dir = (transform.basis * Vector3(move_input.x, 0, move_input.y)).normalized()

# Set velocity
velocity = move_dir * max_speed
```

Adding Jumping Functionality

Next, we need to enable our player character to jump. But before that, we need to apply gravity to our player character.

Applying Gravity

To apply gravity, we need to reduce our Y velocity whenever the player character is not on the floor. We increase our gravity over time using the delta time, which means we are increasing our gravity per second. This is similar to real life, where an object's velocity increases over time when it falls.

Here is how we can apply gravity:

```
# Apply gravity
if not is_on_floor():
    velocity.y -= gravity * delta
```

Implementing Jumping

To implement jumping, we need to check two things: if our jump button was pressed and if we are standing on the floor. We don't want to be able to jump in the air. If these conditions are met, we set our Y velocity to be equal to our jump force.

Here is how we can implement jumping:

```
# Jumping
if Input.is_action_pressed("jump") and is_on_floor():
```



```
velocity.y = jump_force
```

However, there is a problem. Even though we are setting our Y velocity, our movement code is overriding that velocity vector by setting it to be our move direction times max speed. And in our move direction line, the Y is basically zero.

To solve this, we should not override our velocity but set the X and the Z individually. We set the Y velocity in the jumping section, and we don't want to override that at all.

```
# Set X and Z velocity
velocity.x = move_dir.x * max_speed
velocity.z = move_dir.z * max_speed
```

Now, our player character can move relative to the camera's forward direction, jump, and is affected by gravity.

In the next lesson, we will look at how we can implement running for our player character. Stay tuned!



In this lesson, we will continue building our player controller script in Godot by implementing the running movement. We previously set up our player to move relative to the direction of our camera. Now, we will add the ability to run when the shift key is held down.

Setting Up the Run

After calculating the move input and move direction, we will set our ``is_running`` variable. This variable will check if the sprint action (holding down the shift key) is being performed. If it is, ``is_running`` will be true; if not, it will be false.

```
is_running = Input.is_action_pressed("sprint")
```

We then need to adjust our velocity code. Currently, we are multiplying it by our max speed, but we want to change this to be a variable. This is because if we are running (holding down shift), we want to move faster, and if we're not running, we want to move slower.

We will create a variable called ``target_speed`` and set it to our max speed by default. We will then multiply our velocity by ``target_speed`` instead of max speed. If we are running, we will change ``target_speed`` to our ``max_run_speed``.

```
var target_speed = max_speed
if is_running:
    target_speed = max_run_speed
velocity *= target_speed
```

Limiting Running Direction

At this point, we can run at max speed in all directions, which isn't realistic. We want to limit the running movement so that we can't move backwards when running and have limited left and right movement. We will do this using the dot product.

The dot product is a calculation we can perform with two vectors. It returns a value between 1 and -1. If the two vectors are pointing in the same direction, the result is 1. If they are pointing in opposite directions, the result is -1. If they are at a 90-degree angle from each other, the value is 0.

We will calculate the dot product of our move direction and our forward direction (``transform.basis.z``). We will then multiply our move direction by this dot product, which will prevent us from running to the left or right. However, this would cause us to move forward when trying to run backward, so we will clamp our dot product between 0 and 1 to prevent this.

```
var run_dot = -move_dir.dot(transform.basis.z)
run_dot = clamp(run_dot, 0.0, 1.0)
move_dir *= run_dot
```

Gradual Speed Ramp

Our current movement is quite static, meaning we move at max speed instantly when a move key is



pressed. In most games, there is a gradual speed ramp – you start from zero speed and accelerate over time. Similarly, when you let go of the move key, you decelerate rather than stopping instantly. We will implement this acceleration and deceleration in our next lesson.

In summary, we have added running functionality to our player controller. The player can now run when the shift key is held down, and the running movement is limited to prevent unrealistic movements. In the next lesson, we will improve the movement further by adding acceleration and deceleration, as well as air control.



The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

In this lesson, we will be enhancing our player movement in the Godot game engine by adding acceleration, braking, and air acceleration. These elements will make the character movement feel more natural and enjoyable for the player.

Setting Up Variables

Firstly, recall that we have already created variables for acceleration, braking, and air acceleration. These variables will be used in the following way:

- Acceleration: This will be used when we start moving. It will help us transition smoothly to our maximum speed instead of immediately moving at max speed.
- Braking: This will be used when we let go of our inputs and want to stop moving. We can adjust this value to have a slow or fast stop.
- Air Acceleration: This will determine how fast we can move when we are off the ground and in the air. We usually want less control in the air than on the ground, so we set this to a lower value.

Implementing Acceleration, Braking, and Air Acceleration

To implement these features, we need to create a new variable called 'current smoothing'. This variable will be changed based on the state of our player. If the player is in the air, it will be set to 'air acceleration'. If the player is not pressing anything, it will be set to 'braking'. Otherwise, it will be set to 'acceleration'.

```
var current_smoothing = acceleration

if not is_on_floor():
    current_smoothing = air_acceleration
elif not move_direction:
    current_smoothing = braking
```

After setting the current smoothing, we create a variable called 'target velocity', which is our move direction multiplied by our target speed. Then, we use a Lerp function to smoothly transition from our current X and Z velocities to our target velocities at a rate of current smoothing:

```
var target_vel = move_direction * target_speed

velocity.x = lerp(velocity.x, target_vel.x, current_smoothing * delta)
velocity.z = lerp(velocity.z, target_vel.z, current_smoothing * delta)
```

The delta is multiplied to make the smoothing frame independent. This means the smoothing will occur at the same rate, regardless of the frame rate.

Testing the Implementation

After implementing acceleration, braking, and air acceleration, you can run your game to see the



effects. You should notice that your character does not immediately move at max speed when you press a key, and when you let go of the keys, there is a gradual stop. When in the air, the character movement is more floaty.

Adjusting the Values

You can adjust the values of acceleration and braking to make the effects more apparent. For example, you can set acceleration to 15 and braking to 5. This will make the acceleration have a smoother ramp, and the stopping speed will be more gradual.

Adding a Quality of Life Feature

As a quality of life feature, we can make it so that the mouse becomes visible when we press the escape key, and becomes invisible again when we press the escape key again. This allows us to easily exit the game window without having to use Alt+F4 every time.

The following code has been updated, and differs from the video:

Instead of using `get_mouse_mode()` and `set_mouse_mode()`, we can directly access the property `mouse_mode` to get and set it.

```
if Input.is_action_just_pressed("ui_cancel"):
    if Input.mouse_mode == Input.MOUSE_MODE_VISIBLE:
        Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
    else:
        Input.mouse_mode = Input.MOUSE_MODE_VISIBLE
```

In conclusion, adding acceleration, braking, and air acceleration can greatly enhance the feel of the player movement in your game. Remember, you can tweak these values to achieve the exact movement feel you want for your game.



The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

Our player controller is now complete and functional! Let's have a quick review of it, going over what each of the different aspects control.

In the `_ready` function, we are altering our [mouse mode](#) to be captured. This means we are locking the mouse to the game window and disabling the cursor visual.

```
func _ready():
    # Lock the mouse
    Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
```

Physics Process Function

Movement

We start by applying gravity to our velocity, specifically modifying the Y axis as that represents the up and down direction. We are subtracting gravity x delta which reflects the real-world formula of gravity which is 9.81 meters per second per second.

```
# Apply gravity
if not is_on_floor():
    velocity.y -= gravity * delta
```

When detecting a jump, we are checking for 2 things:

- Is the jump action currently pressed down?
- Are we standing on the floor?

If so, then we are setting our Y velocity to be jump force, which will send us up into the air.

```
# Jumping
if Input.is_action_pressed("jump") and is_on_floor():
    velocity.y = jump_force
```

We next calculate our movement direction. This is first done by calculating an [input vector based on our 4 movement actions](#). This will return a vector2 such as this: (-1, 1) for if we have the W and A pressed down (moving forward and left).

We then calculate our movement direction vector. This takes the `move_input` and multiplies it by our facing direction to get our move direction relative to that. Finally, we normalize the vector to prevent moving faster at an angle.

```
# Movement
var move_input = Input.get_vector("move_left", "move_right", "move_forward", "move_backward")
var move_dir = (transform.basis * Vector3(move_input.x, 0, move_input.y)).normalized()
```



To calculate sprinting:

- We set **is_running** to true or false depending on if the sprint action is held down.
- We create a **target_speed** variable which represents how fast we want to move this frame and set it to our default.
- If we are running, update our `target_speed`.
 - Then, calculate a dot product based on our movement direction and forward facing direction. This returns a number from -1 to 1. 1 if the two vectors are pointing in the same direction, -1 if they're opposite, 0 if they are at a 90 degree angle and so on.
 - This is then used to prevent us from running backwards.

```
is_running = Input.is_action_pressed("sprint")
```

```
var target_speed = max_speed
```

```
if is_running:  
    target_speed = max_run_speed  
    var run_dot = -move_dir.dot(transform.basis.z)  
    run_dot = clamp(run_dot, 0.0, 1.0)  
    move_dir *= run_dot
```

Before we start adjusting our movement velocity, we first need to calculate smoothing. This will determine how fast we accelerate or decelerate our velocity.

- If we are in the air, our movement is more floaty.
- If we have no buttons held down, we want to use the braking value to stop us.

```
var current_smoothing = acceleration
```

```
if not is_on_floor():  
    current_smoothing = air_acceleration  
elif not move_dir:  
    current_smoothing = braking
```

Then finally, we calculate our **target_vel** which is our desired move velocity.

- This is then **lerped** towards from our current velocity at the **current_smoothing** rate.
- Last but not least, the velocity is then applied to our player (move, check collisions, etc) by calling the **move_and_slide** function.

```
var target_vel = move_dir * target_speed
```

```
velocity.x = lerp(velocity.x, target_vel.x, current_smoothing * delta)  
velocity.z = lerp(velocity.z, target_vel.z, current_smoothing * delta)
```

```
move_and_slide()
```

Camera Look

At the bottom of the function, we are running the logic for our camera rotation based on mouse

movement.

- We first rotate our player around their Y axis based on the horizontal mouse input.
- We then rotate the camera up and down (X axis) based on the vertical mouse input.
- But there is a problem here! we don't want the camera to go all the way up and around. We want to prevent it from looking too far up and too far down.
 - To fix this, we simply **clamp** the axis between -1.5 and 1.5 (the radian rotation).
- Then set our look input to zero at the end of the frame.

```
# Camera Look
```

```
rotate_y(-camera_look_input.x * look_sensitivity)
camera.rotate_x(-camera_look_input.y * look_sensitivity)
camera.rotation.x = clamp(camera.rotation.x, -1.5, 1.5)
camera_look_input = Vector2.ZERO
```

Mouse Locking

When playing our game, there may be times when we want to unlock our mouse in order to click on something off-screen. To implement this, we are detecting when the *ui_cancel* input action has been pressed (the escape key).

- If our mouse is currently visible and unlocked, lock it.
- Otherwise, if our mouse is currently locked, then unlock it.

The following code has been updated, and differs from the video:

Instead of using `get_mouse_mode()` and `set_mouse_mode()`, we can directly access the property `mouse_mode` to get and set it.

```
# Mouse
if Input.is_action_just_pressed("ui_cancel"):
    if Input.mouse_mode == Input.MOUSE_MODE_VISIBLE:
        Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
    else:
        Input.mouse_mode = Input.MOUSE_MODE_VISIBLE
```

Unhandled Input function

_unhandled_input gets called when we create an input event such as keyboard press, mouse click, movement, etc.

Here, we are checking to see if the event was mouse motion related (moving the mouse). If so, we set the **camera_look_input** vector to be that motion, which represents how far and what distance our mouse has moved this frame.

```
func _unhandled_input(event):
    if event is InputEventMouseMotion:
        camera_look_input = event.relative
```




In this lesson, we will begin implementing an interaction system in Godot. This system will allow us to look at an object, display a prompt on the screen, and interact with it. We'll start by creating nodes for our player using a Raycast 3D.

Setting up the Player Controller

First, we need to set up our player controller. Go to the player controller and add a new child node to the Camera 3D, named Raycast 3D. You can think of Raycast 3D as a laser that can point in a direction and fire off. We can choose how far we want to fire it and it can return information about whatever it is hitting.

By default, the raycast is pointing down. However, we want it to point forward. So, in the inspector, set the target position to be zero on the x, zero on the y, and negative two on the z. Now, you can see that the blue line is pointing forwards.

Rename this raycast to "interaction controller".

Adding the Interaction Prompt

Next, we need to add the interaction prompt. This is the text that will appear on the screen when we are hovering over an interactable object. It will give us a prompt so we can see what the interaction is going to do.

Add a child node to the interaction controller and create a label. This label will switch us into 2D mode because we are now working with UI. Increase the size of the box that appears.

At the top of the viewport, there's a small circular green button. This is where we can choose our anchor presets. Since our text box will be in the middle of our screen, select the center preset. This will move the anchor points down to the center, ensuring that the text box maintains its central position no matter how we resize our screen.

Next, add some example text to the label. For instance, you might have a square bracket, the key to interact (which could be "E"), and a description such as "Pickup item". Rename this label to "interaction prompt".

Finally, adjust the alignment and font size of the text. Set the horizontal alignment to be center, vertical to be center, and increase the font size to around 24.

By default, we want the text to be empty because we don't want it to appear on the screen when we're not interacting with something.

Creating the Interaction Scripts

Now, it's time to create two scripts. The first script will be called "interactable object", and the second will be "interaction controller".

Let's start with the "interactable object" script. This script will be attached to all the objects we want to interact with.

```
class_name InteractableObject
extends Node3D
```

```
@export var interact_prompt : String
@export var can_interact : bool = true
```



```
func _interact():  
    print("Override this function.")
```

We're giving it a class name of "InteractableObject" because we want to access this node of a specific type in our other script. The variables in this script include an "interact prompt" of type string, which will show to the player when they hover over the object, and a boolean "can interact", which determines if the player can interact with the object at a certain time.

The "_interact()" function is a placeholder that will be overridden with specific interaction logic in derived classes. This script will act as the parent class, and any children (doors, objects, lights, etc.) will be inheriting this script and having access to its variables and functions.

Next, let's move to the "interaction controller" script.

```
extends RayCast3D  
  
@onready var interact_prompt_label : Label = get_node("InteractionPrompt")  
  
func _process(delta):  
    var object = get_collider()  
    interact_prompt_label.text = ""  
  
    if object and object is InteractableObject:  
        pass
```

This script extends a RayCast3D node to detect interactive objects in a 3D environment. It gets the collider of whatever object it is currently hitting. If the object exists and is of type InteractableObject, then it can continue on with the if statement.

In the next lesson, we will continue with this function and create an interactable object to see the interaction system in action.



In this lesson, we will continue building our interaction system in the Godot engine by creating an interactable object. This object will have a function that will be called when the player interacts with it. We will also set up an interaction prompt that will appear on the screen when the player is looking at an interactable object. Let's get started.

Building the Interaction Controller

The first thing we need to do is to modify our interaction controller script. The interaction controller is responsible for checking if the player is looking at an interactable object and calling the interact function on that object when the player presses the interact key.

Here's the code for our interaction controller:

```
extends RayCast3D

@onready var interact_prompt_label : Label = get_node("InteractionPrompt")

func _process(delta):
    var object = get_collider()
    interact_prompt_label.text = ""

    if object and object is InteractableObject:
        if object.can_interact == false:
            return

        interact_prompt_label.text = "[E] " + object.interact_prompt

    if Input.is_action_just_pressed("interact"):
        object._interact()
```

In this script, we first get a reference to the object that the player is currently looking at using the `get_collider()` function. We then check if this object is an instance of the `InteractableObject` class and if it can be interacted with. If it can, we update the interaction prompt label with the object's interaction prompt and check if the player has pressed the interact key. If the player has pressed the interact key, we call the object's `_interact()` function.

Creating an Interactable Object

Now that we have our interaction controller set up, let's create an interactable object. For this example, we will create a pedestal that turns on a light when interacted with.

Here's the code for our interactable object:

```
extends InteractableObject

@onready var light_bulb = get_node("LightBulb")

func _interact ():
    light_bulb.visible = true
    can_interact = false
```



In this script, we first get a reference to the light bulb node. We then override the `_interact()` function. When this function is called, we make the light bulb visible and set `can_interact` to false to prevent further interactions.

Testing the Interaction System

With everything set up, we can now test our interaction system. If everything is working correctly, you should be able to walk up to the pedestal, see the interaction prompt, and turn on the light by pressing the interact key.

This is just a simple example of what you can do with this interaction system. You can easily extend this system to create more complex interactions, such as collecting items, solving puzzles, or interacting with NPCs. The possibilities are endless!

Let's have a deeper look into how our interaction controller works and the different aspects that make it up.

How it Works

The interaction controller works by firing a raycast from the center of the camera to whatever direction it's facing. We do this by utilizing a [RayCast3D](#) node. We set this up in the editor, positioning it, defining the direction and distance. Attached to this node is the **InteractionController** script, where we run the logic inside of the **_process** function:

We start by getting the object the RayCast3D is hitting with the **get_collider()** function.

```
var object = get_collider()
```

We also set the interact prompt text to be empty, because we do not yet know if we're hovering over something viable.

```
interact_prompt_label.text = ""
```

The rest of the code for this script is contained within this if statement. We are checking two things:

1. Does the object contain a value that is not null?
2. Does the object contain an **InteractableObject** script or script that derives from that class?

If so, then we can interact with it.

```
if object and object is InteractableObject:
```

The next step involves returning and ending the function if we cannot interact with this object.

```
if object.can_interact == false:  
    return
```

If we can, then we want to notify the player, so we update the UI label to represent that – displaying the object's interact prompt.

```
interact_prompt_label.text = "[E] " + object.interact_prompt
```

And then finally, if the interact action has been pressed, we want to call the object's **_interact** function.

```
if Input.is_action_just_pressed("interact"):  
    object._interact()
```

Interactable Objects

So we have our interact controller working, but what about the actual *things* we'll be interacting with? These are defined by the **InteractableObject** script. If a node has this attached, then it can be interacted with. Alternatively, we can create a new script which extends from `InteractableObject` and that too will be recognised.

This script has 2 variables:

- **interact_prompt** - The text displayed on screen when hovering over the object.
- **can_interact** - Can we currently interact with this object? We can enable/disable this boolean whenever.

```
@export var interact_prompt : String
@export var can_interact : bool = true
```

Then we have just 1 function, **_interact**. Right now, this function simply prints out a message, but if we extend from this class when creating a new script, we can do what's known as overriding a function. This essentially means creating a function with the same name as one in the base class. So whenever that function gets called, the overridden function you just defined will be called instead.

```
func _interact():
    print("Override this function.")
```

You'll notice this in the **PedestalInteractable** script, which we extend from `InteractableObject`.

We have re-created the **_interact** function, so that when it gets called from the controller script, this one will execute with all its custom code relative to the pedestal.

```
func _interact ():
    light_bulb.visible = true
    can_interact = false
```

[Learn more about GDScripting and extending scripts here.](#)

Creating a New Interactable Object

To create a new interactable object, setup the node object you wish to use and attach a new script to it.

First, make sure that the script extends from **InteractableObject**.

```
extends InteractableObject
```

Then, override the **_interact** function and add in your own custom code that you wish to execute.

```
func _interact ():
```



your code here

And that is it!



Congratulations on successfully completing the course on creating a walking simulator using the Godot engine. This course equipped you with the knowledge to create two systems, the play controller and the interaction controller, which can be incorporated into any project you undertake in Godot.

The Play Controller

The play controller is an essential component of your walking simulator. It grants the player the ability to move around the game environment, run, jump, and manipulate the camera for different views. We also added a feature for smooth movement, allowing for a more tactile experience when the player is on the ground as opposed to when they are in the air.

You can save the play controller node as a scene and insert it into any project you desire. However, remember to also transfer the input maps, as these are vital for controlling player movement.

The Interaction Controller

The interaction controller is another crucial system we created during this course. It allows the player to interact with objects within the game. When the player hovers over an interactable object, a prompt is displayed. By pressing 'E', the player can engage with the object.

This system can be used to create a variety of interactive components in your game, such as doors, items, light switches, and more. It introduces a single function call on the interactable object, offering you the flexibility to create whatever you want.

About Zenva

Zenva is an online learning academy, serving over a million students worldwide. We offer a broad range of courses, catering to beginners as well as those who are seeking to learn something new. Our courses are versatile, providing a variety of learning methods to suit your needs. You can choose to watch online video tutorials, read the lesson summary, or follow along with the instructor using the course files.

Thank you for choosing to learn with us. We wish you the best of luck with your future projects in Godot.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

PlayerController.gd

Found in Project/PlayerController

This code defines the `PlayerController` class for a 3D character in a game, which extends the `CharacterBody3D` class in the Godot Engine. It manages player movement and camera control, including running, jumping, gravity effects, and mouse look sensitivity, with customizable properties for tuning these behaviors. The `\_physics\_process` function updates the character's movement based on user input, applies gravity, and adjusts the camera angle accordingly. The `\_unhandled\_input` function captures mouse motion for camera look control.

```
class_name PlayerController
extends CharacterBody3D

@export_group("Movement")
@export var max_speed : float = 4.0
@export var acceleration : float = 20.0
@export var braking : float = 20.0
@export var air_acceleration : float = 4.0
@export var jump_force : float = 5.0
@export var gravity_modifier : float = 1.5
@export var max_run_speed : float = 6.0
var is_running : bool = false

@export_group("Camera")
@export var look_sensitivity : float = 0.005
var camera_look_input : Vector2

@onready var camera : Camera3D = get_node("Camera3D")
@onready var gravity : float = ProjectSettings.get_setting("physics/3d/default_gravity") * gravity_modifier

func _ready():
    # Lock the mouse
    Input.mouse_mode = Input.MOUSE_MODE_CAPTURED

func _physics_process(delta):
    # Apply gravity
    if not is_on_floor():
        velocity.y -= gravity * delta

    # Jumping
    if Input.is_action_pressed("jump") and is_on_floor():
        velocity.y = jump_force

    # Movement
    var move_input = Input.get_vector("move_left", "move_right", "move_forward", "move_back")
```



```
var move_dir = (transform.basis * Vector3(move_input.x, 0, move_input.y)).normalized()

is_running = Input.is_action_pressed("sprint")

var target_speed = max_speed

if is_running:
    target_speed = max_run_speed
    var run_dot = -move_dir.dot(transform.basis.z)
    run_dot = clamp(run_dot, 0.0, 1.0)
    move_dir *= run_dot

var current_smoothing = acceleration

if not is_on_floor():
    current_smoothing = air_acceleration
elif not move_dir:
    current_smoothing = braking

var target_vel = move_dir * target_speed

velocity.x = lerp(velocity.x, target_vel.x, current_smoothing * delta)
velocity.z = lerp(velocity.z, target_vel.z, current_smoothing * delta)

move_and_slide()

# Camera Look
rotate_y(-camera_look_input.x * look_sensitivity)
camera.rotate_x(-camera_look_input.y * look_sensitivity)
camera.rotation.x = clamp(camera.rotation.x, -1.5, 1.5)
camera_look_input = Vector2.ZERO

# Mouse
if Input.is_action_just_pressed("ui_cancel"):
    if Input.get_mouse_mode() == Input.MOUSE_MODE_VISIBLE:
        Input.set_mouse_mode(Input.MOUSE_MODE_CAPTURED)
    else:
        Input.set_mouse_mode(Input.MOUSE_MODE_VISIBLE)

func _unhandled_input(event):
    if event is InputEventMouseMotion:
        camera_look_input = event.relative
```

InteractableObject.gd

Found in Project/PlayerController

This code defines a `InteractableObject` class, inheriting from `Node3D`, which is intended to represent objects in a 3D environment that can be interacted with. It includes an exported string variable for interaction prompts and a boolean to toggle the ability to interact. The `_interact()` function is a placeholder meant to be overridden with specific interaction logic in derived classes.



```
class_name InteractableObject
extends Node3D

@export var interact_prompt : String
@export var can_interact : bool = true

func _interact():
    print("Override this function.")
```

InteractactionController.gd

Found in Project/PlayerController

This script extends a `RayCast3D`` node to detect interactive objects in a 3D environment. During each frame, if the ray collides with an `InteractableObject`` and interaction is allowed (`can_interact``), it updates an on-screen label with an interaction prompt (e.g., “[E] Interact with object”). If the player presses the interact key (bound to the “interact” action), the object’s `_interact()`` method is called to perform the interaction.

```
extends RayCast3D

@onready var interact_prompt_label : Label = get_node("InteractionPrompt")

func _process(delta):
    var object = get_collider()
    interact_prompt_label.text = ""

    if object and object is InteractableObject:
        if object.can_interact == false:
            return

        interact_prompt_label.text = "[E] " + object.interact_prompt

    if Input.is_action_just_pressed("interact"):
        object._interact()
```

PedestalInteraction.gd

Found in the Project root folder

This piece of code defines a subclass of `InteractableObject`` that, when interacted with, turns on a light bulb and disables further interactions by setting `can_interact`` to false.

```
extends InteractableObject

@onready var light_bulb = get_node("LightBulb")

func _interact():
    light_bulb.visible = true
    can_interact = false
```